

RoboSim / RoboAnalyzer: Project Overview (Part 1 of 2)

This document represents the first half of an in-depth exploration of the RoboSim (RoboAnalyzer) project. This part focuses on the **Theoretical Foundations**, **Mathematical Models**, and the **Backend Architecture** that powers the robot simulation.

1. Project Purpose & Scope

RoboSim is a comprehensive desktop application built to simulate, visualize, and analyze robotic manipulators. It allows users to interact with predefined industrial robots (like the UR5, SCARA, Stanford Arm, PUMA 560, and ABB IRB 120) or construct their own custom robots by defining Denavit-Hartenberg (DH) parameters.

Core capabilities include:

- * **Forward Kinematics (FK):** Calculating the 3D position of the End-Effector (tool tip) based on joint angles.
- * **Inverse Kinematics (IK):** Calculating the required joint angles to reach a specific 3D Cartesian target (XYZ).
- * **Real-time Simulation:** Simulating telemetry data and rendering the robot's movement in a responsive 3D viewport.
- * **Trajectory Planning:** Queuing up waypoints (via UI or CSV) and smoothly animating the robot through the path.

2. Theoretical Foundation & Mathematics

Robotics simulation relies heavily on linear algebra and spatial geometry. RoboSim implements these concepts from scratch using NumPy.

2.1. Denavit-Hartenberg (DH) Parameters

The project uses the **Standard Denavit-Hartenberg (DH)** convention to model the robot mathematically. A robot is modeled as a series of rigid links connected by joints (either *revolute* for rotation or *prismatic* for sliding).

Each joint is defined by four parameters:

1. θ (**Theta**): Joint angle (rotation around the previous Z-axis). For revolute joints, this is the variable.
2. d (**Offset**): Link offset (translation along the previous Z-axis). For prismatic joints, this is the variable.
3. a (**Link Length**): Length of the common normal (translation along the new X-axis).
4. α (**Twist Angle**): Link twist (rotation around the new X-axis).

2.2. Forward Kinematics (Transformation Matrices)

To find where the End-Effector is in 3D space, the backend calculates a 4×4 Homogeneous Transformation Matrix for each joint. These matrices combine rotation and translation.

For a joint i , the matrix T_i is computed as:

$$T_i = RotZ(\theta) \cdot TransZ(d) \cdot TransX(a) \cdot RotX(\alpha)$$

The final position of the End-Effector relative to the base is the sequential multiplication of all joint matrices:

$$T_{end} = T_1 \cdot T_2 \cdot T_3 \cdot \dots \cdot T_n$$

The top-right 3×1 vector in T_{end} contains the exact (X, Y, Z) coordinates.

2.3. Inverse Kinematics (Damped Least Squares)

While Forward Kinematics is a simple multiplication, Inverse Kinematics (IK) is highly complex because multiple joint configurations can reach the same target, or a target might be unreachable.

RoboSim uses a **Numerical Inverse Kinematics** approach called **Damped Least Squares (Levenberg-Marquardt)**.
1. **The Jacobian (J):** The system calculates the Jacobian matrix, which maps joint velocities to End-Effector velocities. It answers the question: “*If I move joint X slightly, how much does the End-Effector move?*”
2. **Iterative Correction:** The algorithm computes the error between the *current* position and the *target* position.
3. **Pseudo-Inverse:** Instead of a pure matrix inversion (which crashes near singularities/gimbal lock), it adds a damping factor (λ) to the Jacobian:

$$\Delta q = J^T (J \cdot J^T + \lambda^2 I)^{-1} \cdot \Delta e$$

4. **Convergence:** It repeatedly updates the joint angles (Δq) until the error (Δe) is below a tiny threshold (e.g., 0.001 meters), meaning the target is reached.

3. Backend Architecture (backend/)

The backend is completely decoupled from the UI, focusing purely on data generation, state management, and math.

3.1. kinematics.py

This is the mathematical brain of the project.
* **DHJoint Data Class:** Represents a single joint, storing its type (`revolute` or `prismatic`), limits (`q_min`, `q_max`), and DH parameters.
* **KinematicChain Class:** * Maintains a list of

DHJoint objects. * **forward_kinematics(angles)**: Iterates through the joints, builds the 4×4 matrices using NumPy, and returns the 3D positions of every joint (so the UI can draw the “bones” of the robot). * **inverse_kinematics(target)**: Implements the Damped Least Squares algorithm described above. It features dynamic step-sizing and early-exit mechanisms if the target is physically out of reach.

3.2. robot_presets.py

A library of real-world industrial robots mathematically modeled using DH parameters. * **Functions like preset_scara(), preset_stanford_arm(), preset_ur5()** return pre-configured KinematicChain objects. * This demonstrates the power of the generic IK solver—because the IK solver is purely mathematical based on the Jacobian, it can solve for *any* of these presets without requiring custom hardcoded formulas for each robot.

3.3. Core Simulation & Telemetry

- **sim_worker.py**: Generates simulated telemetry data (simulating physical hardware). It runs a mathematical loop (like a sine wave generator) to simulate the robot moving in a circle or back-and-forth, emitting data frames at roughly 60Hz.
- **serial_worker.py**: Designed to ingest physical hardware telemetry via USB/Serial using the pyserial library.
- **replay_buffer.py & replay_controller.py**: Act as the “DVR” for the project. They record incoming telemetry frames into a memory buffer and manage a state machine (IDLE, RECORDING, PLAYING, STOPPED) to allow users to pause, scrub, and playback historical robot movements frame-by-frame.